

ME314 Homework 4

Submission instructions

Deliverables that should be included with your submission are shown in **bold** at the end of each problem statement and the corresponding supplemental material. **Your homework will be graded IFF you submit a single PDF, .mp4 videos of animations when requested and a link to a Google colab file that meet all the requirements outlined below.**

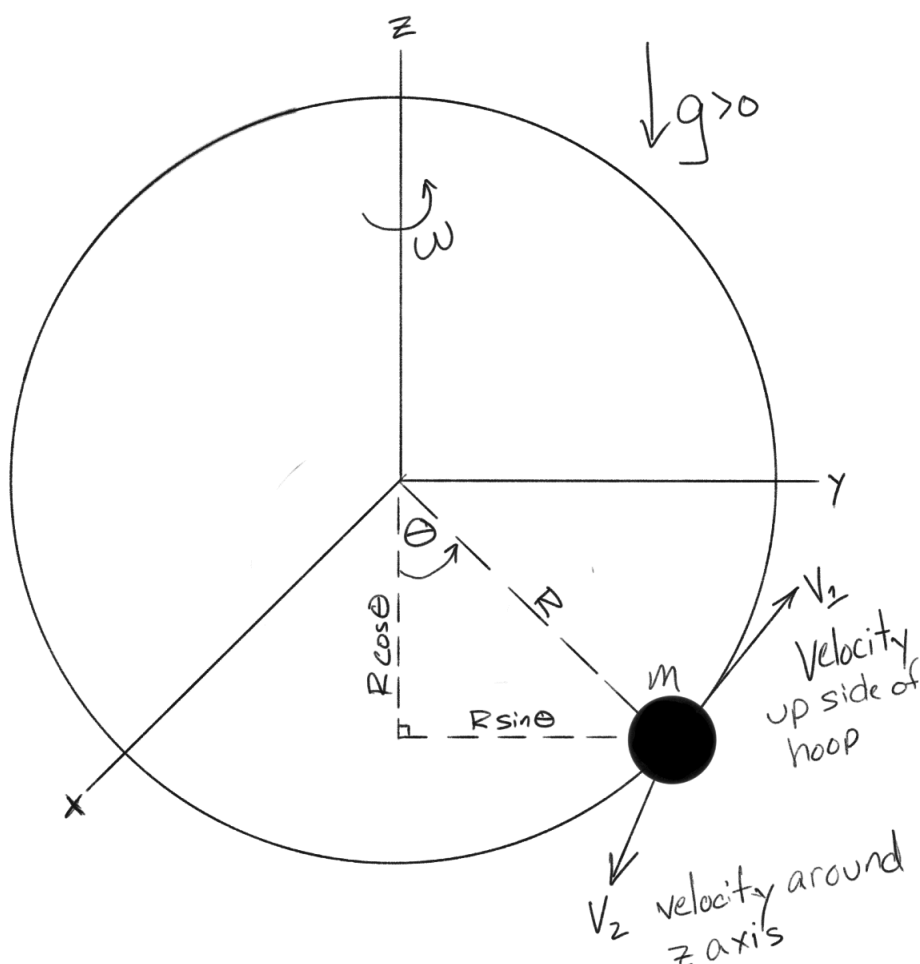
- List the names of students you've collaborated with on this homework assignment.
- Include all of your code (and handwritten solutions when applicable) used to complete the problems.
- Highlight your answers (i.e. **bold** and outline the answers) for handwritten or markdown questions and include simplified code outputs (e.g. `.simplify()`) for python questions.
- Enable Google Colab permission for viewing
- Click Share in the upper right corner
- Under "Get Link" click "Share with..." or "Change"
- Then make sure it says "Anyone with Link" and "Editor" under the dropdown menu
- Make sure all cells are run before submitting (i.e. check the permission by running your code in a private mode)
- Please don't make changes to your file after submitting, so we can grade it!
- Submit a link to your Google Colab file that has been run (before the submission deadline) and don't edit it afterwards!

NOTE: This Juputer Notebook file serves as a template for you to start homework. Make sure you first copy this template to your own Google driver (click "File" -> "Save a copy in Drive"), and then start to edit it.

```
In [1]: import sympy as sym
import numpy as np
import matplotlib.pyplot as plt
```

```
In [2]: #####
# If you're using Google Colab, uncomment this section by selecting the whole section and press
# ctrl+'/' on your keyboard. Run it before you start programming, this will enable the nice
# LaTeX "display()" function for you. If you're using the local Jupyter environment, leave it alone
#####
def custom_latex_printer(exp,**options):
    from google.colab.output._publish import javascript
    url = "https://cdnjs.cloudflare.com/ajax/libs/mathjax/3.1.1/latest.js?config=TeX-AMS_HTML"
    javascript(url=url)
    return sym.printing.latex(exp,**options)
sym.init_printing(use_latex="mathjax", latex_printer=custom_latex_printer)
```

```
In [3]: from IPython.core.display import HTML
display(HTML("<table><tr><td><img src='https://github.com/MuchenSun/ME314pngs/raw/master/dynhoop2.png' width=528.4'"))
```



Problem 1 (20pts)

Take the bead on a hoop example shown in the image above, model it using a torque input τ (about the vertical z axis) instead of a velocity input ω . You will need to add a configuration variable ψ that is the rotation about the z axis, so that the system configuration vector is $q = [\theta, \psi]$. Use Python's SymPy package to compute the equations of motion for this system in terms of θ, ψ .

Hint 1: Note that this should be a Lagrangian system with an external force.

Turn in: A copy of code used to symbolically solve for the equations of motion, also include the code outputs, which should be the equations of motion.

```
In [4]: # Define the variables and parameters
t, g, m, R, tau = sym.symbols(r't, g, m, R, \tau')

th = sym.Function(r'\theta')(t)
psi = sym.Function(r'\psi')(t)

th_dot = th.diff(t)
psi_dot = psi.diff(t)

KE = (m*R**2*th_dot**2 + m*R**2*psi_dot**2*(sym.sin(th)**2))/2
PE = m*g*R*(1-sym.cos(th))

lagrangian = KE - PE
lagrangian = sym.simplify(lagrangian)

# Display Lagrangian
print('\033[1mLagrangian of the system: ')
display(lagrangian)
print("\n")

# Wrap the lagrangian into a SymPy Matrix
L = sym.Matrix([lagrangian])
# display(L)

q = sym.Matrix([th, psi])
qdot = q.diff(t)
qddot = qdot.diff(t)

# Compute Jacobian
dLdq = L.jacobian(q)
dLdqdot = L.jacobian(qdot)

# print('\033[1mEuler Lagrangian Equations')
EL = dLdqdot.diff(t) - dLdq

# for exp, n in zip(EL, range(1,3)):
#     print("\033[1mEL Equation {}".format(n))
#     display(sym.Eq(exp, 0))
#     print("\n")

# LHS of the EL equations
lhs = sym.Matrix([exp for exp in EL])

# RHS of the EL equations
rhs = sym.Matrix([0, tau])

# Define the equations
eqns = sym.Eq(lhs, rhs)
print('\033[1mEuler Lagrangian Equations')
display(eqns)

solns = sym.solve(eqns, qddot, dict=True)

for sol in solns:
    print('\n\033[1mSolution: ')
    for q in qddot:
        display(sym.simplify(sym.Eq(q, sol[q])))
    print("\n")
```

Lagrangian of the system:

$$\frac{Rm \left(R \sin^2 (\theta(t)) \left(\frac{d}{dt} \psi(t) \right)^2 + R \left(\frac{d}{dt} \theta(t) \right)^2 + 2g (\cos (\theta(t)) - 1) \right)}{2}$$

Euler Lagrangian Equations

$$\begin{bmatrix} R^2m \frac{d^2}{dt^2} \theta(t) - \frac{Rm \left(2R \sin (\theta(t)) \cos (\theta(t)) \left(\frac{d}{dt} \psi(t) \right)^2 - 2g \sin (\theta(t)) \right)}{2} \\ R^2m \sin^2 (\theta(t)) \frac{d^2}{dt^2} \psi(t) + 2R^2m \sin (\theta(t)) \cos (\theta(t)) \frac{d}{dt} \psi(t) \frac{d}{dt} \theta(t) \end{bmatrix} = \begin{bmatrix} 0 \\ \tau \end{bmatrix}$$

Solution:

$$\frac{d^2}{dt^2}\theta(t) = \frac{\left(R \cos(\theta(t)) \left(\frac{d}{dt}\psi(t)\right)^2 - g\right) \sin(\theta(t))}{R}$$

$$\frac{d^2}{dt^2}\psi(t) = \frac{-R^2 m \sin(2\theta(t)) \frac{d}{dt}\psi(t) \frac{d}{dt}\theta(t) + \tau}{R^2 m \sin^2(\theta(t))}$$

Problem 2 (30pts)

Consider a point mass in 3D space under the forces of gravity and a radial spring from the origin. The system's Lagrangian is:

$$L = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2 + \dot{z}^2) - \frac{1}{2}k(x^2 + y^2 + z^2) - mgz$$

Consider the following rotation matrices, defining rotations about the z , y , and x axes respectively:

$$R_\theta = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad R_\psi = \begin{bmatrix} \cos \psi & 0 & \sin \psi \\ 0 & 1 & 0 \\ -\sin \psi & 0 & \cos \psi \end{bmatrix}, \quad R_\phi = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix}$$

and answer the following three questions:

1. Which, if any, of the transformations $q_\theta = R_\theta q$, $q_\psi = R_\psi q$, or $q_\phi = R_\phi q$ keeps the Lagrangian fixed (invariant)? Is this invariance global or local?
2. Use small angle approximations to linearize your transformation(s) from the first question. The resulting new transformation(s) should have the form $q_\epsilon = q + \epsilon G(q)$. Compute the difference in the Lagrangian $L(q_\epsilon, \dot{q}_\epsilon) - L(q, \dot{q})$ through this/these transformation(s).
3. Apply Noether's theorem to determine a conserved quantity. What does this quantity represent physically? Is there any rationale behind its conservation?

You can solve this problem by hand or use Python's SymPy to do the symbolic computation for you.

Hint 1: For question (1), try to imagine how this system looks. Even though the x , y , and z axes seem to have the same influence on the system, rotation around some axes will influence the Lagrangian more than others will.

Hint 2: Global invariance here means for any magnitude of rotation the Lagrangian will remain fixed.

Turn in: A scanned (or photograph from your phone or webcam) copy of your hand written solution. You can also use $\text{\textit{LATEX}}$. If you use SymPy, then you just need to include a copy of code and the code outputs, with notes that explain why the code outputs can answer the questions.

ME 314 Theory of Machine Dynamics

Homework 4

October 29, 2024

Phani Kiran Vadlamani

Collaborators : NA

Problem 2

Point mass in 3D space, forces of gravity and spring

$$L = \frac{1}{2} m (\dot{x}^2 + \dot{y}^2 + \dot{z}^2) - \frac{1}{2} k (x^2 + y^2 + z^2) - mgz$$

i) $q = [x \ y \ z]^T$

$$q_\theta = R_\theta q$$

$$q_\theta = \begin{bmatrix} x \cos \theta - y \sin \theta \\ x \sin \theta + y \cos \theta \\ z \end{bmatrix}$$

Representing $\sin \theta$ as s_θ and $\cos \theta$ as c_θ

Similarly $\sin^2 \theta$ as $\tilde{s}_\theta$ and $\cos^2 \theta$ as $\tilde{c}_\theta$.

Notation applicable throughout the question

$$L_\theta = \frac{1}{2} m \left[\frac{d}{dt}(x \cos \theta - y \sin \theta) \right]^2 + \frac{1}{2} m \left[\frac{d}{dt}(x \sin \theta + y \cos \theta) \right]^2 + \frac{1}{2} m \left[\frac{d}{dt}(z) \right]^2 - \frac{1}{2} k \left[(x \cos \theta - y \sin \theta)^2 + (x \sin \theta + y \cos \theta)^2 + (z)^2 \right] - mgz$$

$$L_0 = \frac{1}{2}m \left[(\dot{x}\cos\theta - \dot{y}\sin\theta)^2 + (\dot{x}\sin\theta + \dot{y}\cos\theta)^2 + \dot{z}^2 \right] - \frac{1}{2}k \left[(x\cos\theta - y\sin\theta)^2 + (x\sin\theta + y\cos\theta)^2 + z^2 \right] - mgz$$

$$L_0 = \frac{1}{2}m \left[\dot{x}^2 (\tilde{s}^2\theta + \tilde{c}^2\theta) + \dot{y}^2 (\tilde{s}^2\theta + \tilde{c}^2\theta) + \dot{z}^2 \right] - \frac{1}{2}k \left[x^2 (\tilde{s}^2\theta + \tilde{c}^2\theta) + y^2 (\tilde{s}^2\theta + \tilde{c}^2\theta) + z^2 \right] - mgz$$

$$L_0 = \frac{1}{2}m \left[\dot{x}^2 + \dot{y}^2 + \dot{z}^2 \right] - \frac{1}{2}k \left[x^2 + y^2 + z^2 \right] - mgz$$

$L_0 = L \quad \forall \theta$ i.e Transformation is invariant $\forall \theta$

Thus $q_\theta = R_\theta q$ is global invariant

$$q_\psi = R_\psi q$$

$$q_\psi = \begin{bmatrix} \cos\psi & 0 & \sin\psi \\ 0 & 1 & 0 \\ -\sin\psi & 0 & \cos\psi \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$L_\psi = \frac{1}{2}m \left[\frac{d}{dt}(x\cos\psi + z\sin\psi) \right]^2 + \frac{1}{2}m \left[\frac{d}{dt}(y) \right]^2 + \frac{1}{2}m \left[\frac{d}{dt}(-x\sin\psi + z\cos\psi) \right]^2 - \frac{1}{2}k \left[(x\cos\psi + z\sin\psi)^2 + y^2 + (-x\sin\psi + z\cos\psi)^2 \right] - mg(-x\sin\psi + z\cos\psi)$$

$$L_\psi = \frac{1}{2}m (\dot{x}^2 + \dot{y}^2 + \dot{z}^2) - \frac{1}{2}k(x^2 + y^2 + z^2) - mg(-x\sin\psi + z\cos\psi) \\ (L_\psi \neq L)$$

Taking very small values of ψ

$$\sin \psi \approx \psi \quad \text{and} \quad \cos \psi \approx 1$$

$$L_\psi = \frac{1}{2} m (\dot{x}^2 + \dot{y}^2 + \dot{z}^2) - \frac{1}{2} k (x^2 + y^2 + z^2) - mg(-x\psi + z)$$

$$L_\psi = L + \underbrace{mgx\psi}$$

Cannot guarantee to be zero even for very small ψ

$\therefore q_\psi = R_\psi q$ does not keep the Lagrangian invariant

$$q_\phi = R_\phi q$$

$$q_\phi = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$L_\phi = \frac{1}{2} m \left[\frac{d}{dt}(x) \right]^2 + \frac{1}{2} m \left[\frac{d}{dt}(y \cos \phi - z \sin \phi) \right]^2 + \frac{1}{2} m \left[\frac{d}{dt}(y \sin \phi + z \cos \phi) \right]^2 \\ - \frac{1}{2} k \left[x^2 + (y \cos \phi - z \sin \phi)^2 + (y \sin \phi + z \cos \phi)^2 \right] \\ - mg(y \sin \phi + z \cos \phi)$$

$$L_\phi = \frac{1}{2} m (\dot{x}^2 + \dot{y}^2 + \dot{z}^2) - \frac{1}{2} k (x^2 + y^2 + z^2) - mg(y \sin \phi + z \cos \phi)$$

$$L_\phi \neq L$$

Taking small values of ϕ

$$L\phi = L - \underline{mg y \phi}$$

cannot guarantee to zero even for small ϕ

$\therefore q_\phi = Rq$ does not keep the Lagrangian invariant

2)

$$q_\epsilon = q + \epsilon Q(q)$$

To approximate the transformation locally and find $Q(q)$

We take derivative of $R \cdot (x \ y \ z)^T$

(i) when $R = R_0$

$$q_\epsilon = \begin{bmatrix} x - y\epsilon \\ x\epsilon + y \\ z \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} + \epsilon \begin{bmatrix} -y \\ x \\ 0 \end{bmatrix}$$

$\uparrow$
 $Q(q)$

$$\begin{aligned} \sin \epsilon &\approx \epsilon \\ \cos \epsilon &\approx 1 \end{aligned}$$

$$\begin{aligned} L(\dot{q}_\epsilon, q_\epsilon) &= \frac{1}{2} m \left[(\dot{x} - \dot{y}\epsilon)^2 + (\dot{x}\epsilon + \dot{y})^2 + \dot{z}^2 \right] - \\ &\quad \frac{1}{2} k \left[(x - y\epsilon)^2 + (x\epsilon + y)^2 + z^2 \right] - mgz \\ &= \frac{1}{2} m \left[\dot{x}^2 (1 + \epsilon^2) + \dot{y}^2 (1 + \epsilon^2) + \dot{z}^2 \right] - \\ &\quad \frac{1}{2} k \left[x^2 (1 + \epsilon^2) + y^2 (1 + \epsilon^2) + z^2 \right] - mgz \\ &= L(q, \dot{q}) + \frac{1}{2} m (\dot{x}^2 \epsilon^2 + \dot{y}^2 \epsilon^2) - \frac{1}{2} k (x^2 \epsilon^2 + y^2 \epsilon^2) \end{aligned}$$

$$L(\dot{q}_\epsilon, q_\epsilon) = L(q, \dot{q}) + \epsilon^2 \left(\frac{1}{2} m (\dot{x}^2 + \dot{y}^2) - \frac{1}{2} k (x^2 + y^2) \right)$$

$$L(\dot{q}_\epsilon, q_\epsilon) - L(q, \dot{q}) = \frac{\epsilon^2}{2} \left[m(\dot{x}^2 + \dot{y}^2) - k(x^2 + y^2) \right]$$

(ii) when $R = R_\psi$

$$q_\epsilon = \begin{bmatrix} x + z\epsilon \\ y \\ -x\epsilon + z \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} + \epsilon \begin{bmatrix} z \\ 0 \\ -x \end{bmatrix} \leftarrow q(q)$$

$$L(q_\epsilon, \dot{q}_\epsilon) = \frac{1}{2} m \left[(\dot{x} + \dot{z}\epsilon)^2 + \dot{y}^2 + (-\dot{x}\epsilon + \dot{z})^2 \right] - \frac{1}{2} k \left((x + z\epsilon)^2 + y^2 + (-x\epsilon + z)^2 \right) - mg(-x\epsilon + z)$$

$$L(q_\epsilon, \dot{q}_\epsilon) = \frac{1}{2} m \left[x^2 (1 + \epsilon^2) + y^2 + z^2 (1 + \epsilon^2) \right] - \frac{1}{2} k \left[x^2 (1 + \epsilon^2) + y^2 + z^2 (1 + \epsilon^2) \right] - mgz + mgx\epsilon$$

$$L(q_\epsilon, \dot{q}_\epsilon) = L(q, \dot{q}) + \epsilon^2 \frac{m^2}{2} (\dot{x}^2 + \dot{z}^2) - \epsilon^2 \frac{k}{2} (x^2 + z^2) + mgx\epsilon$$

$$L(q_\epsilon, \dot{q}_\epsilon) - L(q, \dot{q}) = \frac{\epsilon^2 m^2}{2} (\dot{x}^2 + \dot{z}^2) - \frac{\epsilon^2 k}{2} (x^2 + z^2) + mgx\epsilon$$

(iii) when $R = R_\phi$

$$q_\epsilon = \begin{bmatrix} x \\ y - z\epsilon \\ y\epsilon + z \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} + \epsilon \begin{bmatrix} 0 \\ -z \\ y \end{bmatrix} \leftarrow q(q)$$

$$L(q_\epsilon, \dot{q}_\epsilon) - L(q, \dot{q}) = \frac{\epsilon^2 m^2}{2} (\dot{y}^2 + \dot{z}^2) - \frac{\epsilon^2 k}{2} (y^2 + z^2) - \dots mg y \epsilon$$

[Calculations are same as above.
Replace x with $-y$ in above result]

3) When $R = R_\theta$

$$L_\epsilon - L = \frac{\epsilon^2}{2} \left[m(\dot{x}^2 + \dot{y}^2) - k(x^2 + y^2) \right]$$

$$\left. \frac{dL}{d\epsilon} \right|_{\epsilon=0} = 0 \quad \text{Thus } G(q) = \begin{bmatrix} -y \\ x \\ 0 \end{bmatrix} \text{ is a valid transformation}$$

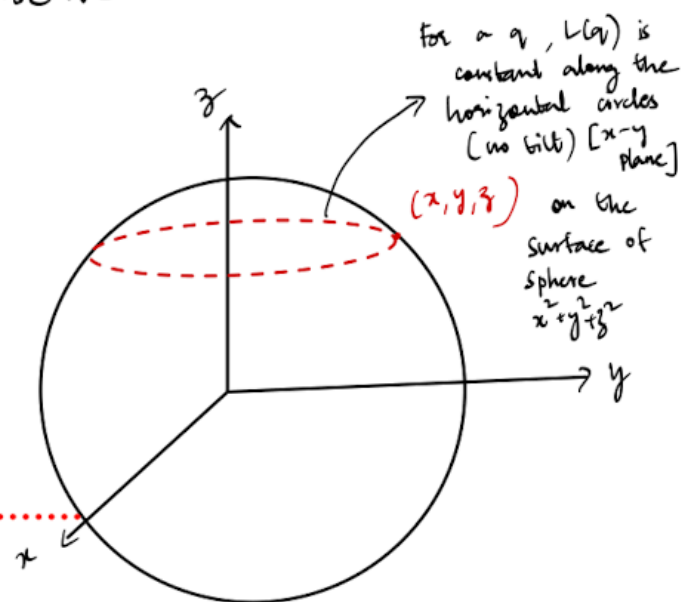
$$\begin{aligned} \frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}} G(q) \right) &= \frac{d}{dt} \left(\begin{bmatrix} m_{ix} & m_{iy} & m_{iz} \end{bmatrix} \begin{bmatrix} -y \\ x \\ 0 \end{bmatrix} \right) = 0 \\ &= \frac{d}{dt} \left(-m_{ix} y + m_{iy} x \right) = 0 \end{aligned}$$

Thus $-m_{ix} y + m_{iy} x$ is conserved

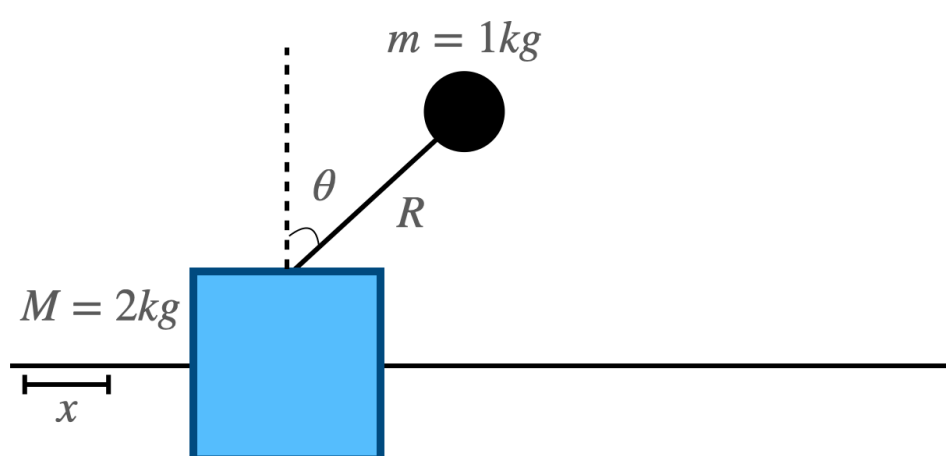
This is the z-component of angular momentum

This is conserved because there is no torque at origin due to the spring force and torque only in x axis and y axis due to gravity i.e. zero torque about z-axis

∴ z-axis Angular momentum is constant and conserved



In [5]: `from IPython.core.display import HTML
display(HTML("<table><tr><td><img src='https://github.com/atulleyaylor/ME314Figures/raw/main/hw4problem3.png' width"))`



Problem 3 (20pts)

For the inverted cart-pendulum system in Homework 1 (feel free to make use of the provided solutions), compute the conserved momentum using Nöther's theorem. Plot the momentum for the same simulation parameters and initial conditions. Taking into account the conserved quantities, what is the minimal number of states in the cart/pendulum system that can vary? (Hint: In some coordinate systems it may look like all the states are varying, but if you choose your coordinates cleverly fewer states will vary.)

Turn in: A copy of code used to calculate the conserved quantity and your answer to the question. You don't need to turn in equations of motion, but you need to include the plot of the conserved quantity evaluated along the system trajectory.

```
In [6]: t, m, M, R, g = sym.symbols('t, m, M, R, g')

# define states and time derivatives
x = sym.Function('x')(t)
th = sym.Function('theta')(t)

q = sym.Matrix([x, th])

xdot = x.diff(t)
thdot = th.diff(t)
qdot = q.diff(t)

# xddot = xdot.diff(t)
# thddot = thdot.diff(t)
qddot = qdot.diff(t)

# Compute pendulum's velocity in x and y direction
# This is under the "world frame", from where we measure
# The position of the cart as x(t)
pen_xdot = xdot + R*thdot*sym.cos(th)
pen_ydot = -R*thdot*sym.sin(th)

# compute Lagrangian
KE = 0.5*M*xdot**2 + 0.5*m*(pen_xdot**2 + pen_ydot**2)
PE = m*g*R*sym.cos(th)

lagrangian = KE - PE
print('\033[1mLagrangian: ')
display(sym.simplify(L))
print("\n")

# Compute the derivatives
L = sym.Matrix([lagrangian])

# Compute Jacobian
dLdq = L.jacobian(q)
dLdqdot = L.jacobian(qdot)

EL = dLdqdot.diff(t) - dLdq

# LHS of the EL equations
lhs = sym.Matrix([exp for exp in EL])

# RHS of the EL equations
rhs = sym.Matrix([0, 0])

# Define the equations
eqns = sym.Eq(lhs, rhs)
print('\033[1mEuler Lagrangian Equations')
display(sym.simplify(eqns))

# Solve the equations
solns = sym.solve(eqns, qddot, dict=True)

# Substitute constants into symbolic solutions
subs_dict = {m:1, M:2, g:9.8, R:1}

c_sub_solns = [{q: soln[q].subs(subs_dict) for soln in solns for q in qddot}]

for c_sub_soln in c_sub_solns:
    print('\n\033[1mSolution: ')
    for q in qddot:
        display(sym.simplify(sym.Eq(q, c_sub_soln[q])))
    print("\n")

# Lambdifying the constant substituted solutions for theta1_ddot and theta2_ddot
state_vec = [x, qdot[0], th, qdot[1]]
lambdified_solns = [sym.lambdify(state_vec, c_sub_soln[q]) for c_sub_soln in c_sub_solns for q in qddot]
xddot_func, thddot_func = lambdified_solns
```

Lagrangian:

$$\left[\frac{Rm \left(R \sin^2(\theta(t)) \left(\frac{d}{dt} \psi(t) \right)^2 + R \left(\frac{d}{dt} \theta(t) \right)^2 + 2g(\cos(\theta(t)) - 1) \right)}{2} \right]$$

Euler Lagrangian Equations

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1.0M \frac{d^2}{dt^2} x(t) + 1.0m \left(-R \sin(\theta(t)) \left(\frac{d}{dt} \theta(t) \right)^2 + R \cos(\theta(t)) \frac{d^2}{dt^2} \theta(t) + \frac{d^2}{dt^2} x(t) \right) \\ 1.0Rm \left(R \frac{d^2}{dt^2} \theta(t) - g \sin(\theta(t)) + \cos(\theta(t)) \frac{d^2}{dt^2} x(t) \right) \end{bmatrix}$$

Solution:

$$\frac{d^2}{dt^2} x(t) = \frac{\left(-9.8 \cos(\theta(t)) + 1.0 \left(\frac{d}{dt} \theta(t) \right)^2 \right) \sin(\theta(t))}{\sin^2(\theta(t)) + 2}$$

$$\frac{d^2}{dt^2}\theta(t) = \frac{\left(-\cos(\theta(t))\left(\frac{d}{dt}\theta(t)\right)^2 + 29.4\right)\sin(\theta(t))}{\sin^2(\theta(t)) + 2}$$

```
In [7]: # G(q) is [1, 0]
# Calculating the conserved momentum
p = dLdqdot.dot([1, 0])

# Substituting the constants
print("\033[1mConserved Momentum")
display(p)
p = p.subs(subs_dict)
display(sym.simplify(p))

# Lambdifying the momentum function
p_func = sym.lambdify(state_vec, p)
```

Conserved Momentum

$$1.0M\frac{d}{dt}x(t) + 0.5m\left(2R\cos(\theta(t))\frac{d}{dt}\theta(t) + 2\frac{d}{dt}x(t)\right)$$

$$1.0\cos(\theta(t))\frac{d}{dt}\theta(t) + 3.0\frac{d}{dt}x(t)$$

```
In [8]: # define functions to be used for trajectory plotting
def integrate(f, xt, dt):
    """
    This function takes in an initial condition x(t) and a timestep dt,
    as well as a dynamical system f(x) that outputs a vector of the
    same dimension as x(t). It outputs a vector x(t+dt) at the future
    time step.

    Parameters
    =====
    dyn: Python function
    derivate of the system at a given step x(t),
    it can considered as \dot{x}(t) = func(x(t))

    xt: NumPy array
    current step x(t)

    dt: step size for integration

    Return
    =====
    new_xt:
    value of x(t+dt) integrated from x(t)
    """
    k1 = dt * f(xt)
    k2 = dt * f(xt+k1/2.)
    k3 = dt * f(xt+k2/2.)
    k4 = dt * f(xt+k3)
    new_xt = xt + (1/6.) * (k1+2.0*k2+2.0*k3+k4)
    return new_xt

def simulate(f, x0, tspan, dt, integrate):
    """
    This function takes in an initial condition x0, a timestep dt,
    a time span tspan consisting of a list [min_time, max_time],
    as well as a dynamical system f(x) that outputs a vector of the
    same dimension as x0. It outputs a full trajectory simulated
    over the time span of dimensions (xvec_size, time_vec_size).

    Parameters
    =====
    f: Python function
    derivate of the system at a given step x(t),
    it can considered as \dot{x}(t) = func(x(t))

    x0: NumPy array
    initial conditions

    tspan: Python list
    tspan = [min_time, max_time], it defines the start and end
    time of simulation

    dt:
    time step for numerical integration

    integrate: Python function
    numerical integration method used in this simulation

    Return
    =====
    x_traj:
```

```

        simulated trajectory of x(t) from t=0 to tf
        """
        N = int((max(tspan)-min(tspan))/dt)
        x = np.copy(x0)
        tvec = np.linspace(min(tspan),max(tspan),N)
        xtraj = np.zeros((len(x0),N))
        xtraj[:,0] = x0

        for i in range(1,N):
            xtraj[:,i]=integrate(f,x,dt)
            x = np.copy(xtraj[:,i])

        return xtraj

# define extended dynamics for the cart pendulum system
def cart_pendulum_dyn(x):
    """
        System dynamics function (extended)

        Parameters
        =====
        s: NumPy array
        s = [x, th, xdot, thdot] is the extended system
        state vector, including the position and
        the velocity of the particle

        Return
        =====
        sdot: NumPy array
        time derivative of input state vector,
        sdot = [xdot, thdot, xddot, thddot]

    """
    x_k = x[0]
    xdot_k = x[1]
    theta_k = x[2]
    thetadot_k = x[3]

    xddot_k = xddot_func(x_k, xdot_k, theta_k, thetadot_k)
    thetaddot_k = thddot_func(x_k, xdot_k, theta_k, thetadot_k)

    return np.array([xdot_k, xddot_k, thetadot_k, thetaddot_k])

# Simulation parameters
t0 = 0
tf = 10
dt = 0.1
N = int((tf - t0)/dt)
tvec = np.linspace(t0, tf, N)

# initial conditions
x0 = np.array([0, 0, 0.1, 0])

# simulate the trajectory
x_traj = simulate(cart_pendulum_dyn, x0, [t0, tf], dt, integrate)

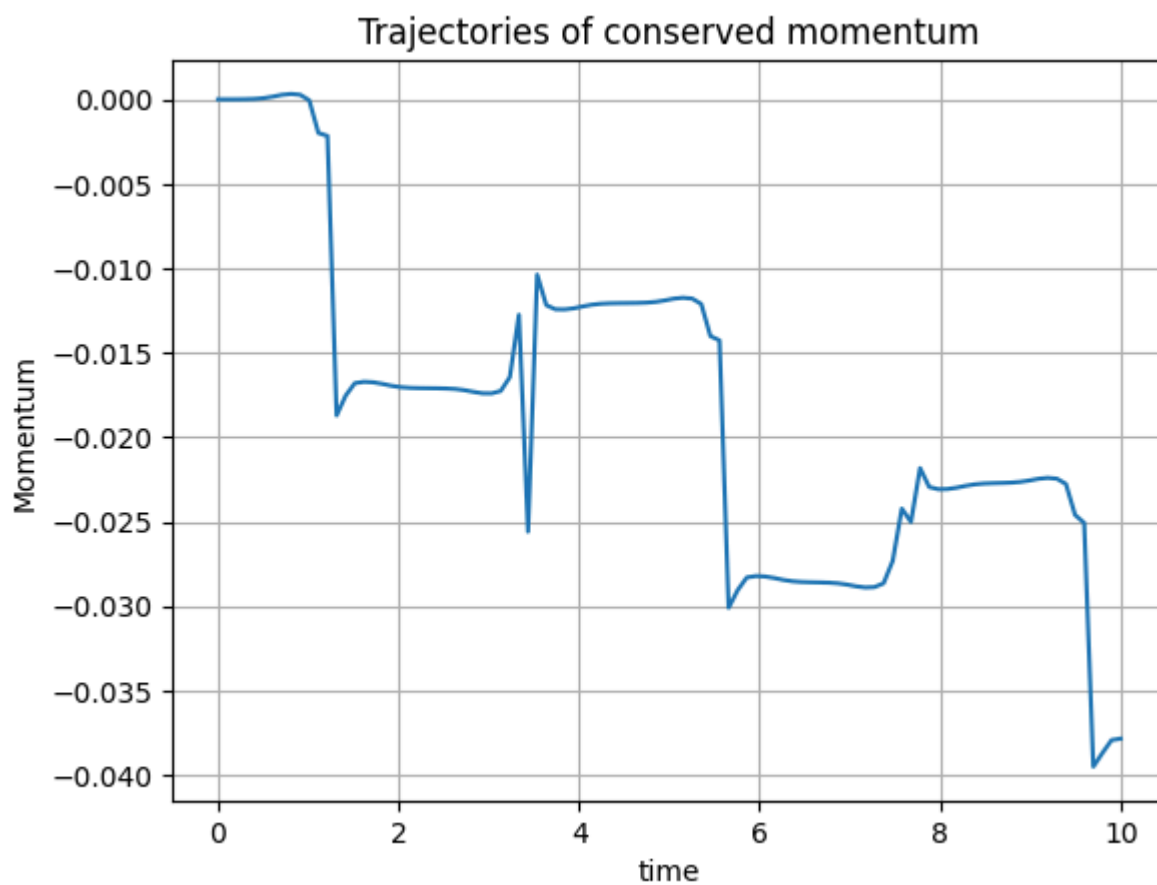
# plot x(t) and th(t)
# plt.plot(tvec, x_traj[0,:])
# plt.plot(tvec, x_traj[2,:])
# plt.title(r'Trajectories of x(t) and $\theta(t)$')
# plt.xlabel("time")
# plt.legend(('x(t)', r'$\theta(t)$'), loc='upper right')
# plt.show()

```

```

In [9]: p_traj = p_func(x_traj[0,:], x_traj[1,:], x_traj[2,:], x_traj[3,:])
# plot p(t)
plt.plot(tvec, p_traj)
plt.title(r'Trajectories of conserved momentum')
plt.xlabel("time")
plt.ylabel("Momentum")
plt.grid()
plt.show()

```



The above pendulum-cart system can be modelled using only state $\theta(t)$. Since horizontal angular momentum is conserved (due to no external forces in x-direction), $x(t)$ can be written in terms of $\theta(t)$ as follows:

Horizontal momentum is conserved

$$M \dot{x} + m R \cos \theta \cdot \dot{\theta} + m \dot{x} = P_x$$

$$\dot{x} (M+m) = P_x - m R \frac{d(\sin \theta)}{dt}$$

$$\dot{x} = \frac{P_x}{m+M} - \frac{m}{M+m} R \frac{d}{dt}(\sin \theta)$$

$$x(t) = \frac{P_x t}{m+M} - \frac{m R}{m+M} \sin \theta + C$$

where C_1 and C_2
are constants

P_x : Horizontal momentum

C : Integration constant

Problem 4 (30 pts)

Using the same inverted cart pendulum system, add a constraint such that the pendulum follows the path of a parabola with a vertex of $(1, 0)$.

Then, simulate the system using x and θ as the configuration variables for $t \in [0, 15]$ with $dt = 0.01$. The constants are $M = 2, m = 1, R = 1, g = 9.8$. Use the initial conditions $x = 0, \theta = 0, \dot{x} = 0, \dot{\theta} = 0.01$ for your simulation.

You should use the Runge-Kutta integration function provided in previous homework for simulation. Plot the simulated trajectory for x, θ versus time. We have provided an animation function for testing.

Hint 1: You will need the time derivatives of ϕ to solve the system of equations.

Hint 2: Make sure to be solving for λ at the same time as your equations of motion.

Hint 3: Note that if you make your initial condition velocities faster or dt lower resolution, you may not be able to simulate the system because this is challenging constraint.

Turn in: A copy of code used to simulate the system, you don't need to turn in equations of motion, but you need to include the plot of the simulated trajectories.

```
In [10]: from IPython.core.display import HTML
display(HTML("<table><tr><td></script>
                <script>
                    requirejs.config({
                        paths: {
                            base: '/static/base',
                            plotly: 'https://cdn.plot.ly/plotly-1.5.1.min.js?noext',
                        },
                    });
                </script>
            '''))
        configure_plotly_browser_state()
        init_notebook_mode(connected=False)

        #####
        # Getting data from pendulum angle trajectories.
        xcart=traj_array[0]
        ycart = 0.0*np.ones(traj_array[0].shape)
        N = len(traj_array[1])

        xx1=xcart+R*np.sin(traj_array[1])
        yy1=R*np.cos(traj_array[1])

        # Need this for specifying length of simulation

        #####
        # Using these to specify axis limits.
        xm=-4
        xM= 4
        ym=-4
        yM= 4

        #####
        # Defining data dictionary.
        # Trajectories are here.
        data=[
```



```

dict(x=xcart, y=ycart,
    mode='markers', name='Cart Traj',
    marker=dict(color="green", size=2)
),
dict(x=xx1, y=yy1,
    mode='lines', name='Arm',
    line=dict(width=2, color='blue')
),
dict(x=xx1, y=yy1,
    mode='lines', name='Pendulum',
    line=dict(width=2, color='purple')
),

dict(x=xx1, y=yy1,
    mode='markers', name='Pendulum Traj',
    marker=dict(color="purple", size=2)
),
]

#####
# Preparing simulation layout.
# Title and axis ranges are here.
layout=dict(xaxis=dict(range=[xm, xM], autorange=False, zeroline=False, dtick=1),
    yaxis=dict(range=[ym, yM], autorange=False, zeroline=False, scaleanchor = "x", dtick=1),
    title='Cart Pendulum Simulation',
    hovermode='closest',
    updatemenus= [{ 'type': 'buttons',
        'buttons': [{ 'label': 'Play', 'method': 'animate',
            'args': [None, {'frame': {'duration': T, 'redraw': False}}]},
            { 'args': [[None], {'frame': {'duration': T, 'redraw': False}, 'mode': 'i',
            'transition': {'duration': 0}}], 'label': 'Pause', 'method': 'animate'}
        ]
    }
    ])

)

#####
# Defining the frames of the simulation.
# This is what draws the lines from
# joint to joint of the pendulum.
frames=[dict(data=[go.Scatter(
    x=[xcart[k]],
    y=[ycart[k]],
    mode="markers",
    marker_symbol="square",
    marker=dict(color="blue", size=30)),

    dict(x=[xx1[k], xcart[k]],
        y=[yy1[k], ycart[k]],
        mode='lines',
        line=dict(color='red', width=3)
    ),
    go.Scatter(
        x=[xx1[k]],
        y=[yy1[k]],
        mode="markers",
        marker=dict(color="blue", size=12)),

    ]) for k in range(N)]

#####
# Putting it all together and plotting.
figure1=dict(data=data, layout=layout, frames=frames)
iplot(figure1)

```

In [12]: `t, m, M, R, g, lambda_ = sym.symbols(r't, m, M, R, g, \lambda')`

```

# define states and time derivatives
x = sym.Function(r'x')(t)
th = sym.Function(r'\theta')(t)

# Compute pendulum's x and y coordinate
pen_x = x + R*sym.sin(th)
pen_y = R*sym.cos(th)

q = sym.Matrix([x, th])

xdot = x.diff(t)
thdot = th.diff(t)
qdot = q.diff(t)

# xddot = xdot.diff(t)
# thddot = thdot.diff(t)
qddot = qdot.diff(t)

# Model the constraint phi
phi = pen_x + pen_y**2 - 1
P = sym.Matrix([phi])

```



```

phi_dot = phi.diff(t)
phi_ddot = phi_dot.diff(t)

dPdq = P.jacobian(q)

# Compute pendulum's velocity in x and y direction
# This is under the "world frame", from where we measure
# The position of the cart as x(t)
pen_xdot = xdot + R*thdot*sym.cos(th)
pen_ydot = -R*thdot*sym.sin(th)

# compute Lagrangian
KE = 0.5*M*xdot**2 + 0.5*m*(pen_xdot**2 + pen_ydot**2)
PE = m*g*R*sym.cos(th)

lagrangian = KE - PE
print('\033[1mLagrangian: ')
display(sym.simplify(lagrangian))
print("\n")

# Compute the derivatives
L = sym.Matrix([lagrangian])

# Compute Jacobian
dLdq = L.jacobian(q)
dLdqdot = L.jacobian(qdot)

EL_constrained = dLdqdot.diff(t) - dLdq

lhs = []
rhs = []
qnew = []

# Create the lhs and rhs of the constrained EL equations
for l, r in zip(EL_constrained, dPdq):
    lhs.append(l)
    rhs.append(lambda_*r)

# Add the equation ddot(phi) = 0
lhs.append(phi_ddot)
rhs.append(0)

lhs = sym.Matrix(lhs)
rhs = sym.Matrix(rhs)

# Define the equations
eqns = sym.Eq(lhs, rhs)

# Define the equations
eqns = sym.Eq(lhs, rhs)
print('\033[1mEuler Lagrangian Equations')
display(sym.simplify(eqns))

for q in qddot:
    qnew.append(q)
    qnew.append(lambda_)

qnew = sym.Matrix(qnew)

# Solve the equations
solns = sym.solve(eqns, qnew, dict=True)

# Substitute constants into symbolic solutions
subs_dict = {m:1, M:2, g:9.8, R:1}

c_sub_solns = [{q: soln[q].subs(subs_dict) for soln in solns for q in qnew}]

for c_sub_soln in c_sub_solns:
    print('\n\033[1mSolution: ')
    for q in qnew:
        display(sym.simplify(sym.Eq(q, c_sub_soln[q])))
    print("\n")

# Lamdifying the constant substituted solutions for theta1_ddot and theta2_ddot
state_vec = [x, qdot[0], th, qdot[1]]
lamdified_solns = [sym.lamdify(state_vec, c_sub_soln[q]) for c_sub_soln in c_sub_solns for q in qnew]
xddot_constrained_func, thddot_constrained_func, lambda_constrained_func = lamdified_solns

```

Lagrangian:

$$0.5M\left(\frac{d}{dt}x(t)\right)^2 - Rgm \cos(\theta(t)) + 0.5m\left(R^2\left(\frac{d}{dt}\theta(t)\right)^2 + 2R \cos(\theta(t))\frac{d}{dt}\theta(t)\frac{d}{dt}x(t) + \left(\frac{d}{dt}x(t)\right)^2\right)$$

Euler Lagrangian Equations

$$\begin{bmatrix} \lambda \\ R\lambda(-R\sin(2\theta(t)) + \cos(\theta(t))) \\ 0 \end{bmatrix} = \begin{bmatrix} 1.0M\frac{d^2}{dt^2}x(t) + 1.0m\left(-R\sin(\theta(t))\left(\frac{d}{dt}\theta(t)\right)^2 + R\cos(\theta(t))\frac{d^2}{dt^2}\theta(t) + \frac{d^2}{dt^2}x(t)\right) \\ 1.0Rm\left(R\frac{d^2}{dt^2}\theta(t) - g\sin(\theta(t)) + \cos(\theta(t))\frac{d^2}{dt^2}x(t)\right) \\ -R^2\sin(2\theta(t))\frac{d^2}{dt^2}\theta(t) - 2R^2\cos(2\theta(t))\left(\frac{d}{dt}\theta(t)\right)^2 - R\sin(\theta(t))\left(\frac{d}{dt}\theta(t)\right)^2 + R\cos(\theta(t))\frac{d^2}{dt^2}\theta(t) + \frac{d^2}{dt^2}x(t) \end{bmatrix}$$

Solution:

$$\frac{d^2}{dt^2}x(t) = \frac{\left(4.0\sin^4(\theta(t))\left(\frac{d}{dt}\theta(t)\right)^2 - 2.0\sin^3(\theta(t))\left(\frac{d}{dt}\theta(t)\right)^2 - 8.0\sin^2(\theta(t))\left(\frac{d}{dt}\theta(t)\right)^2 + 19.6\sin(\theta(t))\cos(\theta(t)) - 9.8\cos(\theta(t)) + 5.0\left(\frac{d}{dt}\theta(t)\right)^2\right)\sin(\theta(t))}{-12.0\sin^4(\theta(t)) + 8.0\sin^3(\theta(t)) + 11.0\sin^2(\theta(t)) - 8.0\sin(\theta(t)) + 2.0}$$

$$\frac{d^2}{dt^2}\theta(t) = \frac{9.8\sin(\theta(t)) + 0.5\sin(2\theta(t))\left(\frac{d}{dt}\theta(t)\right)^2 - 3.0\sin(4\theta(t))\left(\frac{d}{dt}\theta(t)\right)^2 + 1.0\cos(\theta(t))\left(\frac{d}{dt}\theta(t)\right)^2 + 3.0\cos(3\theta(t))\left(\frac{d}{dt}\theta(t)\right)^2}{-12.0\sin^4(\theta(t)) + 8.0\sin^3(\theta(t)) + 11.0\sin^2(\theta(t)) - 8.0\sin(\theta(t)) + 2.0}$$

$$\lambda = \frac{-0.5(1 - \cos(2\theta(t)))^2\left(\frac{d}{dt}\theta(t)\right)^2 + 2.0\sin(\theta(t))\left(\frac{d}{dt}\theta(t)\right)^2 - 9.8\sin(2\theta(t)) + 14.7\cos(\theta(t)) + 4.0\cos(2\theta(t))\left(\frac{d}{dt}\theta(t)\right)^2 - 14.7\cos(3\theta(t))}{-12.0\sin^4(\theta(t)) + 8.0\sin^3(\theta(t)) + 11.0\sin^2(\theta(t)) - 8.0\sin(\theta(t)) + 2.0}$$

```
In [23]: # define extended dynamics for the cart pendulum system
def constrained_cart_pendulum_dyn(x):
    """
    System dynamics function (extended)

    Parameters
    =====
    s: NumPy array
    s = [x, th, xdot, thdot] is the extended system
    state vector, including the position and
    the velocity of the particle

    Return
    =====
    sdot: NumPy array
    time derivative of input state vector,
    sdot = [xdot, thdot, xddot, thddot]

    """
    x_k = x[0]
    xdot_k = x[1]
    theta_k = x[2]
    thetadot_k = x[3]

    xddot_k = xddot_constrained_func(x_k, xdot_k, theta_k, thetadot_k)
    thetaddot_k = thddot_constrained_func(x_k, xdot_k, theta_k, thetadot_k)

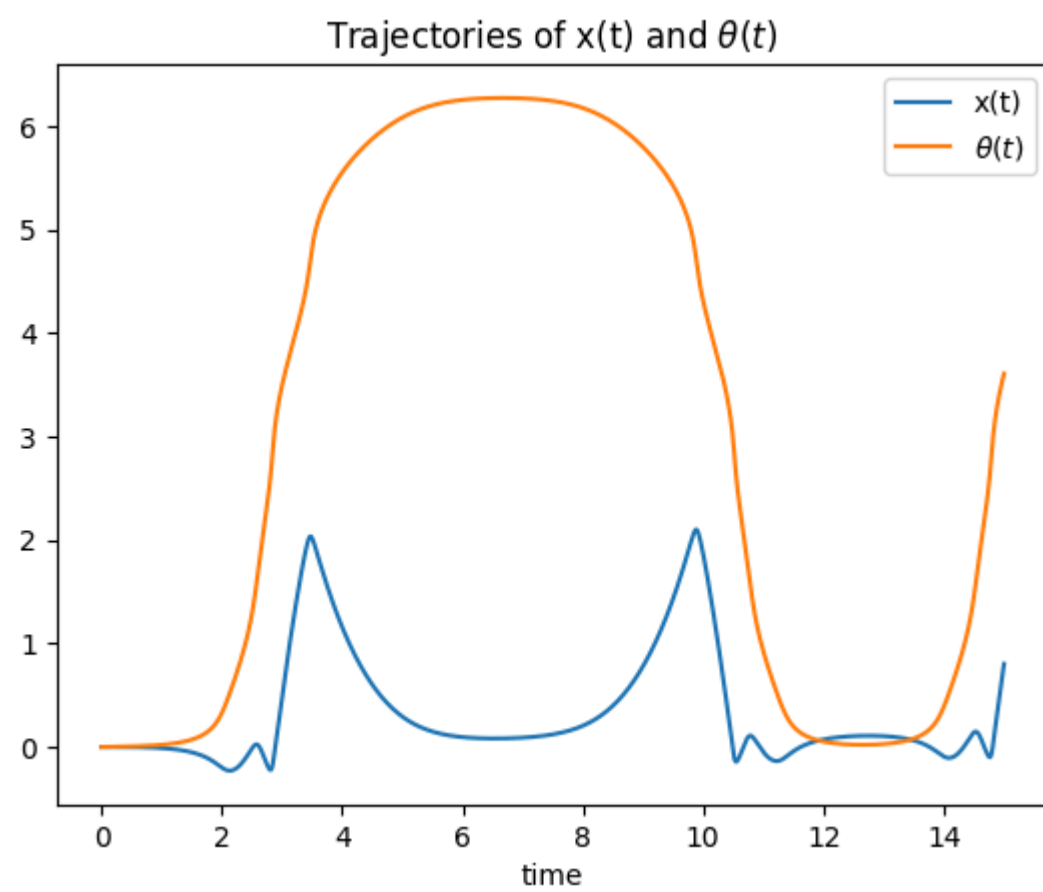
    return np.array([xdot_k, xddot_k, thetadot_k, thetaddot_k])

# Simulation parameters
t0 = 0
tf = 15
dt = 0.01
N = int((tf - t0)/dt)
tvec = np.linspace(t0, tf, N)

# initial conditions
x0 = np.array([0, 0, 0, 0.01])

# simulate the trajectory
x_traj_constrained = simulate(constrained_cart_pendulum_dyn, x0, [t0, tf], dt, integrate)
```

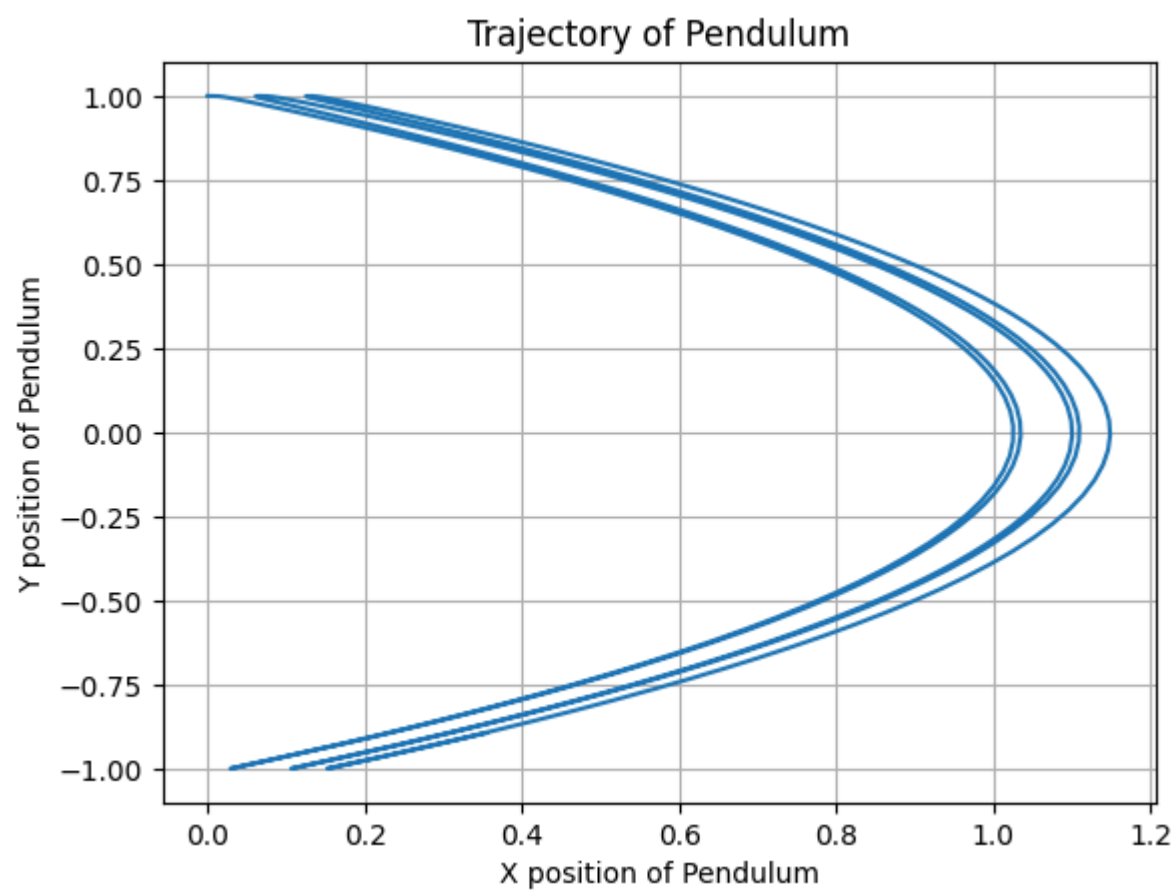
```
In [24]: # plot x(t) and th(t)
plt.plot(tvec, x_traj_constrained[0,:])
plt.plot(tvec, x_traj_constrained[2,:])
plt.title(r'Trajectories of x(t) and $\theta(t)$')
plt.xlabel("time")
plt.legend(('x(t)', r'$\theta(t)$'), loc='upper right')
plt.show()
```



```
In [25]: x_pend = lambda x, theta: x + np.sin(theta)
y_pend = lambda theta: np.cos(theta)

x_pend_traj = x_pend(x_traj_constrained[0,:], x_traj_constrained[2,:])
y_pend_traj = y_pend(x_traj_constrained[2,:])

plt.plot(x_pend_traj, y_pend_traj)
plt.title(r'Trajectory of Pendulum')
plt.xlabel("X position of Pendulum")
plt.ylabel("Y position of Pendulum")
plt.grid()
plt.show()
```



```
In [26]: animate_cart_pend(np.delete(x_traj_constrained, [1, 3], axis=0), R=1, T=15)
```

Cart Pendulum Simulation

Play

Pause

